Linux OS and Scripting
LAB Assignment 4

Name: Vivek B S

Roll No: 261100690034

Subject: Linux OS and Scripting

Part A — Basic grep and Filtering

1: Display all lines containing the word failed.
grep -w "failed" "$LOG"

2: Display all lines containing failed, ignoring case.
grep -i "failed" "$LOG"

3: Display all lines that contain either Failed or Denied.
grep -E "Failed|Denied" "$LOG"

4: Display all lines that do not contain the word INFO.
grep -v "INFO" "$LOG"

5: Display all lines containing the exact word root.
grep -w "root" "$LOG"

6: Display all occurrences of attack along with line numbers.
grep -n "attack" "$LOG"

7: Count the number of lines containing ERROR.
grep -c "ERROR" "$LOG"

8: Display only filenames containing malware.
grep -l "malware" *.log

9: Display the first five lines containing Suspicious.
grep "Suspicious" "$LOG" | head -n 5

10: Display the last five failed-login entries.
grep -i "failed login" "$LOG" | tail -n 5

Part B — Regular Expression Extraction

1: Extract all IPv4 addresses from the log.
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG"

2: Extract all IP addresses belonging to 10.x.x.x network.
grep -Eo '\b10\.([0-9]{1,3}\.){2}[0-9]{1,3}\b' "$LOG"

3: Extract all addresses belonging to 192.168.x.x network.

```
grep -Eo '\b192\.168\.[0-9]{1,3}\.[0-9]{1,3}\b' "$LOG"
```

4: Extract all email addresses.

```
grep -Eo '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$LOG"
```

5: Extract email addresses ending in .com.

```
grep -Eo '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.com\b' "$LOG"
```

6: Extract email addresses with digits in username.

```
grep -Eo '[a-zA-Z0-9._%+-]*[0-9][a-zA-Z0-9._%+-]*@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$LOG"
```

7: Extract all URLs beginning with http:// or https://.

```
grep -Eo 'https?://[^ ]+' "$LOG"
```

8: Extract only HTTPS URLs.

```
grep -Eo 'https://[^ ]+' "$LOG"
```

9: Extract URLs ending with .php.

```
grep -Eo 'https?://[^ ]+\.php\b' "$LOG"
```

10: Extract all CVE identifiers (CVE-YYYY-NNNN).

```
grep -Eo 'CVE-[0-9]{4}-[0-9]{4,7}' "$LOG"
```

11: Extract only CVEs reported for 2025.

```
grep -Eo 'CVE-2025-[0-9]{4,7}' "$LOG"
```

12: Extract all 32-character MD5 hashes.

```
grep -Eo '\b[a-fA-F0-9]{32}\b' "$LOG"
```

13: Extract all 40-character SHA-1 hashes.

```
grep -Eo '\b[a-fA-F0-9]{40}\b' "$LOG"
```

14: Extract all 64-character SHA-256 hashes.

```
grep -Eo '\b[a-fA-F0-9]{64}\b' "$LOG"
```

15: Extract timestamps (YYYY-MM-DD HH:MM:SS).

```
grep -Eo '[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}' "$LOG"
```

Part C — Anchors and Regex Patterns

1: Display all lines beginning with a timestamp.
grep -E '^[0-9]{4}-[0-9]{2}-[0-9]{2}' "$LOG"

2: Display all lines beginning with Suspicious.
grep '^Suspicious' "$LOG"

3: Display lines ending with a numeric value.
grep -E '[0-9]+$' "$LOG"

4: Display lines beginning with ERROR or WARNING.
grep -E '^(ERROR|WARNING)' "$LOG"

5: Display lines whose first character is a digit.
grep '^[0-9]' "$LOG"

6: Find lines ending in .exe.
grep '\.exe$' "$LOG"

7: Find lines with files ending in .exe, .sh, or .pdf.
grep -E '\b[a-zA-Z0-9_-]+\.(exe|sh|pdf)\b' "$LOG"

8: Identify exact word root, excluding rooted.
grep -w "root" "$LOG"

9: Display blank lines in the file.
grep '^$' "$LOG"

10: Display all nonblank lines.
grep -v '^$' "$LOG"

Part D — Linux Filters and Pipelines

1: Extract all IPv4 addresses, sort, and deduplicate.
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort -u

2: Count unique IPv4 addresses.
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort -u | wc-l

3: Extract all IP addresses and display in sorted order.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort -V
```

4: Determine IP addresses appearing more than once.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort | uniq-c | awk '$1 > 1 {print $2}'
```

5: Determine most frequently occurring IP address.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort | uniq-c | sort -nr | head -n 1
```

6: Extract usernames from auth entries (unique).

```
grep -i "user" "$LOG" | sed -E 's/.*user[= ]([a-zA-Z0-9_-]+).*/\1/' | sort -u
```

7: Count auth attempts per username.

```
grep -i "user" "$LOG" | sed -E 's/.*user[= ]([a-zA-Z0-9_-]+).*/\1/' | sort | uniq -c
```

8: Extract HTTP status codes and frequency.

```
grep -Eo 'HTTP/1\.[01]" [0-9]{3}' "$LOG" | awk '{print $2}' | sort | uniq -c
```

9: Display top 5 most frequent IPs.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort | uniq-c | sort -nr | head -n 5
```

10: Display top 3 most frequent usernames.

```
grep -i "user" "$LOG" | sed -E 's/.*user[= ]([a-zA-Z0-9_-]+).*/\1/' | sort | uniq -c | sort -nr | head-n 3
```

11: Extract destination ports in numerical order.

```
grep -Eo 'port [0-9]+' "$LOG" | awk '{print $2}' | sort -n
```

12: Remove duplicate port numbers.

```
grep -Eo 'port [0-9]+' "$LOG" | awk '{print $2}' | sort -nu
```

13: Count unique destination ports.

```
grep -Eo 'port [0-9]+' "$LOG" | awk '{print $2}' | sort -u | wc -l
```

14: Extract file extensions and frequency.

```
grep -Eo '\.[a-zA-Z0-9]{2,4}\b' "$LOG" | sort | uniq -c | sort -nr
```

15: Find top 5 most frequent domains.

```
grep -Eo '([a-zA-Z0-9-]+\.)+[a-zA-Z]{2,}' "$LOG" | sort | uniq -c | sort -nr | head -n 5
```

Part E — Combining grep Conditions

1: Display lines containing IP address and Failed.
grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | grep "Failed"

2: Display lines containing both root and Failed.
grep "root" "$LOG" | grep "Failed"

3: Display lines containing download or upload.
grep -E "download|upload" "$LOG"

4: Display suspicious entries with .exe or .sh.
grep -i "suspicious" "$LOG" | grep -E '\.(exe|sh)\b'

5: Display lines with CVE identifier and IP.
grep -E 'CVE-[0-9]{4}-[0-9]{4,7}' "$LOG" | grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}'

6: Display lines with filename and hash value.
grep -E '\b[a-zA-Z0-9_-]+\.[a-zA-Z0-9]{2,4}\b' "$LOG" | grep-E '\b[a-fA-F0-9]{32,64}\b'

7: Display IP address followed by port.
grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}.*port [0-9]+' "$LOG"

8: Display HTTP requests with URL and 404 status.
grep -E 'https?://[^]+' "$LOG" | grep ' 404 '

9: Display auth failures excluding 127.0.0.1.
grep -i "failed" "$LOG" | grep -v '127\.0\.0\.1'

10: Display attack entries excluding INFO.
grep -i "attack" "$LOG" | grep -v "INFO"

Part F — Negative Matching

1: Extract IPv4 addresses except 127.x.x.x.
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | grep -v '^127\.'

2: Display log entries except successful logins.
grep -iv "success" "$LOG"

3: Exclude INFO, DEBUG, or SUCCESS entries.
grep -Ev "INFO|DEBUG|SUCCESS" "$LOG"

4: Find HTTP requests with status code not 200.
grep 'HTTP/' "$LOG" | grep -v ' 200 '

5: Display suspicious traffic excluding 192.168.x.x.
grep -i "suspicious" "$LOG" | grep -v '192\.168\.'

6: Extract emails excluding company domain.
grep -Eo '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$LOG" | grep -v '@company\.com$'

Part G — Redirection & Combining Filters

1: Extract IPv4 addresses to ips.txt
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" > ips.txt

2: Extract email addresses to emails.txt
grep -Eo '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$LOG" > emails.txt

3: Extract URLs to urls.txt
grep -Eo 'https?://[^]+' "$LOG" > urls.txt

4: Extract hashes to hashes.txt
grep -Eo '\b[a-fA-F0-9]{32,64}\b' "$LOG" > hashes.txt

5: Extract failed logins to failed_login.txt
grep -i "failed login" "$LOG" > failed_login.txt

6: Append suspicious entries to incident_report.txt
grep -i "suspicious" "$LOG" >> incident_report.txt

7: Store unique IP addresses in unique_ips.txt
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort -u > unique_ips.txt

8: Store IP frequency in ip_frequency.txt

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' "$LOG" | sort | uniq-c | sort -nr > ip_frequency.txt
```

9: Redirect output to results.txt and errors to errors.txt

```
grep "ERROR" "$LOG" 1> results.txt 2> errors.txt
```

10: Extract lowercase unique domains to domains.txt

```
grep "Domain accessed" "$LOG" | sed -E 's/.*Domainaccessed:? //I' | tr '[:upper:]' '[:lower:]' | sort -u > domains.txt
```

11: Save unique failed-login IPs to failed_login_ips.txt

```
grep -i "failed login" "$LOG" | grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort -u > failed_login_ips.txt
```

12: Extract failed-login usernames to failed_users.txt

```
grep -i "failed login" "$LOG" | sed -E 's/.*user[= ]([a-zA-Z0-9_-]+).*/\1/' | sort -u > failed_users.txt
```

13: Extract uppercase MD5 hashes to md5_hashes.txt

```
grep -Eo '\b[a-fA-F0-9]{32}\b' "$LOG" | tr '[:lower:]' '[:upper:]' > md5_hashes.txt
```

14: Strip square brackets to clean_log.txt

```
tr -d '[]' < "$LOG" > clean_log.txt
```

15: Convert log file to lowercase normalized_log.txt

```
tr '[:upper:]' '[:lower:]' < "$LOG" > normalized_log.txt
```

16: Find failed-login usernames in uppercase

```
grep -i "failed login" "$LOG" | sed -E 's/.*user[= ]([a-zA-Z0-9_-]+).*/\1/' | tr '[:lower:]' '[:upper:]' | sort-u
```

17: Extract User-Agent field in lowercase

```
grep "User-Agent:" "$LOG" | sed -E 's/.*User-Agent: ?//I' | tr '[:upper:]' '[:lower:]'
```

18: Extract .exe filenames in uppercase

```
grep -i "\.exe" "$LOG" | grep -Eo '\b[a-zA-Z0-9_-]+\.exe\b' | tr '[:lower:]' '[:upper:]'
```